APUNTES COMPLETOS DE DAW

HTML · CSS · JavaScript · PHP · SQL

Guía de consulta rápida con ejemplos comentados, conceptos esenciales y conexiones entre tecnologías.

Objetivo: entender cómo se construye una web desde la estructura visual hasta el almacenamiento de datos, sin entrar en frameworks avanzados.

Material de estudio para practicar, imprimir y consultar durante ejercicios técnicos cuando esté permitido.

Índice y mapa general

HTML estructura el contenido; **CSS** controla su aspecto; **JavaScript** añade interacción; **PHP** ejecuta la lógica en el servidor; **SQL** consulta y modifica los datos.

Flujo típico: usuario → HTML/CSS → evento JavaScript → petición HTTP/JSON → PHP → consulta SQL → respuesta JSON → JavaScript actualiza el DOM.

Carpetas habituales: index.html, css/styles.css, js/app.js, api/db.php y api/tasks.php.

1. HTML — estructura y contenido

HTML usa etiquetas para definir el significado del contenido. Las etiquetas suelen tener apertura y cierre. Los atributos aportan información adicional.

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Mi página</title>
  <link rel="stylesheet" href="css/styles.css">
  <script src="js/app.js" defer></script>
</head>
<body>
  <header><h1>Título principal</h1></header>
  <nav><a href="#contenido">Ir al contenido</a></nav>
  <main id="contenido">
    <section>
      <h2>Sección</h2>
      <p>Un párrafo con <strong>importancia</strong> y <em>énfasis</em>.</p>
    </section>
  </main>
  <footer>Pie de página</footer>
</body>
</html>
```

Etiquetas frecuentes

Etiqueta	Uso
h1–h6	Títulos jerárquicos; h1 suele ser el título principal.
p	Párrafo.
a	Enlace: href indica el destino.
img	Imagen; añade alt descriptivo.
ul / ol / li	Lista sin orden / ordenada / elemento.
table / tr / th / td	Tabla / fila / encabezado / celda.
thead / tbody / caption	Encabezado, cuerpo y título de una tabla.
form / label / input	Formulario, etiqueta y campo.
select / option	Lista desplegable y opción.
textarea	Campo de texto multilínea.
button	Botón; define type: submit, button o reset.
div / span	Contenedores genéricos de bloque / en línea.
header / nav / main / section / article / footer	Estructura semántica.

Listas: ul para viñetas, ol para pasos numerados. Ejemplo: `<ul><li>HTML</li></ul>` .

Tablas: usa th para encabezados y td para datos; organiza con thead y tbody. No uses tablas para maquetar toda una página.

Formularios: relaciona label for="correo" con input id="correo"; name define la clave del dato enviado. required, minlength, maxlength, min, max y type ayudan a validar en el navegador.

```
<form id="user-form">
  <label for="email">Correo</label>
  <input id="email" name="email" type="email" required>
  <button type="submit">Enviar</button>
</form>
```

2. CSS — estilos y diseño

CSS aplica reglas a elementos HTML. Una regla tiene un selector y declaraciones propiedad: valor.

```
/* Selector de etiqueta */
body { font-family: Arial, sans-serif; color: #222; }

/* Selector de clase: se reutiliza */
.card { padding: 16px; margin: 12px; background: white; }

/* Selector de id */
#principal { max-width: 900px; }

/* Descendiente y agrupación */
nav a { text-decoration: none; }
h1, h2 { line-height: 1.2; }

/* Estado del elemento */
a:hover { text-decoration: underline; }
input:focus { outline: 2px solid steelblue; }
```

Propiedades importantes

Grupo	Propiedades frecuentes
Texto	color, font-family, font-size, font-weight, font-style, line-height, text-align, text-decoration, text-transform
Fondo	background, background-color, background-image
Caja	width, max-width, min-height, margin, padding, border, border-radius, box-shadow, box-sizing
Display	display: block / inline / inline-block / none / flex / grid
Flexbox	flex-direction, justify-content, align-items, flex-wrap, gap, flex, align-self
Grid	grid-template-columns, grid-template-rows, gap, place-items
Posición	position: static / relative / absolute / fixed / sticky; top, right, bottom, left, z-index
Listas	list-style, list-style-type, list-style-position
Tablas	border-collapse, table-layout, caption-side
Otros	opacity, overflow, cursor, visibility, transition, transform

Modelo de caja: contenido + padding + borde + margin. Con box-sizing: border-box, width incluye contenido, padding y borde.

Flexbox: ideal para distribuir elementos en una dimensión. justify-content organiza el eje principal; align-items el transversal.

```
.row { display: flex; flex-direction: row; justify-content: space-between;
       align-items: center; flex-wrap: wrap; gap: 12px; }
```

Grid: ideal para filas y columnas.

```
.grid { display: grid; grid-template-columns: repeat(3, 1fr); gap: 16px; }
```

Responsive: media queries adaptan el diseño a pantallas pequeñas.

```
@media (max-width: 600px) {
  .grid { grid-template-columns: 1fr; }
  .container { width: calc(100% - 20px); }
}
```

Unidades: px, %, rem, em, vw, vh. **Pseudoclases:** :hover, :focus, :focus-visible, :checked, :disabled, :first-child, :last-child, :nth-child(even). **Selectores:** *, etiqueta, .clase, #id, A B, A > B, A:hover.

3. JavaScript — lógica del navegador

JavaScript gestiona variables, condiciones, bucles, funciones, arrays, objetos, eventos y peticiones.

```
const appName = "Mi aplicación"; // no reasignar la variable
let counter = 0;                 // sí permite reasignación
counter++;

const text = "hola";   // string
const number = 42;     // number
const active = true;   // boolean
let empty;             // undefined
const nothing = null;  // null

// Operadores: + - * / %, > < >= <=, === !==, && || !
if (counter > 0 && active) {
  console.log("Activo");
} else {
  console.log("Inactivo");
}

// Bucle
for (let i = 0; i < 3; i++) {
  console.log(i);
}

// Función normal y función flecha
function sum(a, b) {
  return a + b;
}
const multiply = (a, b) => a * b;

// Array y métodos
const nums = [1, 2, 3, 4];
const doubled = nums.map(n => n * 2);      // transformar
const even = nums.filter(n => n % 2 === 0); // filtrar
const first = nums.find(n => n > 2);       // primer resultado
const any = nums.some(n => n > 3);          // al menos uno
const total = nums.reduce((acc, n) => acc + n, 0);

// Objeto
const user = { id: 1, name: "Ana" };
console.log(user.name);
```

DOM y eventos

```
const button = document.querySelector("#save");
const output = document.querySelector("#output");

button.addEventListener("click", () => {
  output.textContent = "Guardado";
});

// Formulario: evitar recarga y leer valores
const form = document.querySelector("#form");
form.addEventListener("submit", event => {
  event.preventDefault();
  const data = new FormData(form);
  const email = String(data.get("email") || "").trim();
  console.log(email);
});
```

querySelector busca el primer elemento; querySelectorAll devuelve una colección de coincidencias. addEventListener escucha eventos. classList.add/remove/toggle gestiona clases. textContent inserta texto sin interpretarlo como HTML.

JSON y fetch

```
async function loadData() {
  try {
    const response = await fetch("/api/items.php");
    if (!response.ok) {
      throw new Error("HTTP " + response.status);
    }
    const data = await response.json();
    console.log(data);
```

```
  } catch (error) {
    console.error("Error:", error.message);
  }
}

// Enviar JSON:
fetch("/api/items.php", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ title: "Nueva tarea" })
});
```

JSON.stringify convierte un objeto a texto JSON. response.json() interpreta la respuesta JSON. fetch no rechaza automáticamente una promesa por HTTP 404/500: revisa response.ok. async/await simplifica el trabajo con promesas.

4. PHP — lógica del servidor

PHP se ejecuta en el servidor. Puede recibir datos, validar, consultar la base de datos y generar HTML o JSON.

```
<?php
$name = "Ana";        // variable
$age = 20;
$colors = ["azul", "verde"]; // array indexado
$user = ["id" => 1, "name" => "Luis"]; // asociativo

// Condición
if ($age >= 18) {
    echo "Mayor de edad";
} else {
    echo "Menor de edad";
}

// Bucle
foreach ($colors as $color) {
    echo $color;
}

// Función
function sumNumbers(int $a, int $b): int {
    return $a + $b;
}

// Valores recibidos: siempre validar
$title = trim($_POST["title"] ?? "");
$search = trim($_GET["q"] ?? "");

// Escapar al mostrar texto de usuario en HTML
echo htmlspecialchars($title, ENT_QUOTES | ENT_SUBSTITUTE, "UTF-8");

// Responder JSON
header("Content-Type: application/json; charset=utf-8");
echo json_encode(["success" => true, "message" => "OK"]);
?>
```

Superglobales: $_GET (parámetros URL), $_POST (formulario POST), $_SERVER (método y datos del servidor), $_SESSION (sesión), $_FILES (archivos subidos), $_COOKIE (cookies). $_REQUEST combina fuentes y normalmente es mejor elegir explícitamente la fuente esperada.

Operadores: . concatena cadenas; == compara valores; === compara valor y tipo; ?? usa un valor alternativo cuando el de la izquierda es null/no está definido; === y !== son buenas opciones para comparaciones estrictas.

Seguridad: valida en el servidor, escapa al insertar texto en HTML, usa consultas preparadas para SQL y no muestres errores internos al usuario.

5. SQL — persistencia de datos

```
CREATE DATABASE practica;
USE practica;

CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(150) UNIQUE NOT NULL
);
```

```sql
-- Insertar
INSERT INTO users (name, email)
VALUES ('Ana', 'ana@example.com');

-- Consultar
SELECT * FROM users;
SELECT name FROM users WHERE id = 1;
SELECT * FROM users ORDER BY name ASC LIMIT 10;

-- Modificar y eliminar: ojo con WHERE
UPDATE users SET name = 'Ana López' WHERE id = 1;
DELETE FROM users WHERE id = 1;

-- Agregaciones y grupos
SELECT COUNT(*) AS total FROM users;
SELECT email, COUNT(*) AS total FROM users GROUP BY email;
```

Conceptos: PRIMARY KEY identifica una fila; FOREIGN KEY relaciona tablas; NOT NULL obliga a aportar un valor; UNIQUE impide duplicados; WHERE filtra; ORDER BY ordena; GROUP BY agrupa; JOIN combina tablas. En consultas desde PHP usa PDO prepare/execute con parámetros.

6. PHP + PDO — patrón de consulta segura

```php
<?php
$pdo = new PDO(
    "mysql:host=localhost;dbname=practica;charset=utf8mb4",
    "root",
    "",
    [PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
     PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC]
);

// SELECT
$stmt = $pdo->query("SELECT id, name FROM users");
$users = $stmt->fetchAll();

// INSERT parametrizado
$stmt = $pdo->prepare("INSERT INTO users (name, email) VALUES (?, ?)");
$stmt->execute(["Ana", "ana@example.com"]);

// UPDATE parametrizado
$stmt = $pdo->prepare("UPDATE users SET name = ? WHERE id = ?");
$stmt->execute(["Ana López", 1]);

// DELETE parametrizado
$stmt = $pdo->prepare("DELETE FROM users WHERE id = ?");
$stmt->execute([1]);
?>
```

7. Cómo se conectan las cinco tecnologías

Paso	Tecnología	Qué ocurre
1	HTML/CSS	El usuario ve el formulario y escribe datos.
2	JavaScript	Valida de forma básica y envía una petición fetch.
3	HTTP/JSON	Los datos viajan en el cuerpo de la petición.
4	PHP	Recibe, valida y aplica la lógica del servidor.
5	PDO/SQL	Consulta o modifica la base de datos.
6	PHP/JSON	Devuelve resultado y código HTTP.
7	JavaScript/DOM	Lee la respuesta y actualiza la interfaz.

Métodos HTTP: GET consultar; POST crear; PUT sustituir; PATCH modificar parcialmente; DELETE eliminar. **Estados comunes:** 200 correcto, 201 creado, 204 correcto sin cuerpo, 400 petición inválida, 401 no autenticado, 403 prohibido, 404 no encontrado, 405 método no permitido, 500 error interno.

8. Errores frecuentes y depuración

1. Leer el mensaje de error completo. 2. Revisar archivo y línea. 3. Comprobar datos de entrada y tipos. 4. Usar console.log, depurador y logs de PHP. 5. Probar un cambio cada vez. 6. Volver a probar el caso original y otros casos límite.

Errores comunes: usar = en vez de === al comparar; olvidar return; llamar una función sin argumentos; olvidar preventDefault; no revisar response.ok; olvidar WHERE en UPDATE/DELETE; construir SQL concatenando entradas; usar una ruta de archivo incorrecta; olvidar que PHP requiere servidor.

9. Mini ejercicios de repaso

HTML: crea una tabla con columnas nombre, correo y estado. Añade thead, tbody, th y td.

CSS: centra un contenedor, usa flex para una barra de navegación y añade media query a 600px.

JavaScript: filtra una lista de objetos para devolver solo los activos; transforma el resultado en un array de nombres.

PHP: crea una función que reciba un array de números y devuelva su suma.

SQL: crea una tabla products, inserta dos registros, consulta los de precio mayor de 10, modifica uno y elimina otro.

Integración: explica cómo enviarías un formulario con fetch a PHP, guardarías los datos mediante PDO y devolverías JSON.

10. Lista rápida antes de una prueba

- ■ Sé construir una página con títulos, enlaces, listas, tablas y formularios.
- ■ Distingo class e id y conozco selectores CSS.
- ■ Entiendo flex, grid, el modelo de caja y media queries.
- ■ Sé usar variables, funciones, condicionales, bucles, arrays y objetos.
- ■ Sé seleccionar elementos del DOM y escuchar eventos.
- ■ Sé qué hacen fetch, async/await, JSON.stringify y response.json.
- ■ Entiendo variables, arrays, funciones, $_GET/$_POST y JSON en PHP.
- ■ Sé conectar PDO, preparar consultas y ejecutar CRUD.
- ■ Conozco SELECT, INSERT, UPDATE, DELETE, WHERE, ORDER BY y JOIN.
- ■ Sé explicar el recorrido de un dato entre navegador, servidor y base de datos.

Recuerda: el objetivo no es memorizar todos los ejemplos, sino comprenderlos y poder adaptarlos a otro problema.